

REPORT-CHALLENGE 1(FIX MY BROKEN MODEL)

RACHELAPA MOKOENA KWENA

1.Introduction

This project focused on debugging and fixing a telecom customer complaint classification pipeline.

The original model classified complaints into:

- billing
- network
- service
- fraud

The previous developer claimed the model achieved 94% accuracy and was ready for production. After deployment, the model started misclassifying many complaints. This indicated serious problems in the machine learning pipeline.

The objective of this investigation was to:

- identify the bugs in the pipeline
- explain why the issues occurred
- fix the problems
- rebuild a reliable classification workflow

2. Dataset Overview

The dataset contained 2,000 synthetic South African telecom complaints generated using `generate_dataset.py`.

Category Distribution

Category	Count
Billing	1200
Service	300
Network	300
Fraud	200

The dataset is imbalanced because billing complaints represent 60% of the data. This means accuracy alone is not a reliable evaluation metric.

3.Bug 1: Complaint ID Leakage

What the bug is

The pipeline extracted the first three characters from `complaint_id` and used them as features.

```
df["id_prefix"] = df["complaint_id"].str[:3]
```

The prefixes directly encoded the category:

- BIL -billing

- NET -network
- SVC - service
- FRD - fraud

Why it is a problem

This caused severe data leakage because the model learned category prefixes instead of actual complaint texts, which produced misleadingly high accuracy and would fail on new real-world complaints without category-based IDs.

How I found it

I inspected the feature engineering process and analysed the feature importance results. This revealed that the complaint ID prefixes directly matched the complaint categories, which confirmed the presence of data leakage.

How I fixed it

```
# ONLY use complaint text
# Removed complaint_id Leakage
X = df["clean_text"]

y = df["label"]

return X, y, label_encoder
```

I removed:

I removed id_prefix and all features derived from complaint_id, so the model now relies only on complaint text.

4. Bug 2: TF-IDF Data Leakage

What the bug is

The TF-IDF vectorizer was fitted on the full dataset before train/test splitting.

```
# This Leaks information from test set into training (vocabulary & IDF weights)
vectorizer = TfidfVectorizer(max_features=500, stop_words="english")
tfidf_matrix = vectorizer.fit_transform(df["clean_text"])
```

Why it is a problem

The vectorizer learned vocabulary and word importance statistics from both training and test data. This means information from the test set entered the training process. This leads to inflated performance metrics because the model already sees patterns from the test data during training, instead of learning only from training data.

How I found it

I reviewed the preprocessing pipeline and noticed that fit_transform() occurred before the dataset split.

How I fixed it

I fixed the workflow by splitting the dataset first, fitting TF-IDF only on the training data, and then transforming the test data separately.

5.Bug 3: Incorrect Pipeline Order

What the bug is

Feature engineering and preprocessing were performed before proper train/test separation.

Why it is a problem

Train and test data became contaminated during preprocessing.

This weakened the reliability of evaluation results.

How I found it

I traced the order of operations in the original pipeline.

How I fixed it

I traced the order of operations in the original pipeline. I fixed it by cleaning the text first, splitting the dataset, fitting the vectorizer on training data, then transforming the test data.

6.Bug 4: Misleading Evaluation Metric

What the bug is

The original model relied mainly on accuracy.

Why it is a problem

The dataset is imbalanced.

A model can achieve high accuracy by predicting mostly billing complaints while performing poorly on minority classes.

How I found it

- I inspected the category distribution and evaluation metrics

How I fixed it

I added precision, recall, F1-score and confusion matrix. This produced a more realistic evaluation.

7.Bug 5: Incorrect Label Mapping

What the bug is

The classification report used incorrect class ordering.

Why it is a problem

Performance metrics appeared under the wrong labels. This made interpretation misleading.

How I found it

I compared LabelEncoder ordering and manually hardcoded labels

How I fixed it

I replaced hardcoded labels with:

```
target_names=label_encoder.classes_
```

8. Additional Dataset Issue

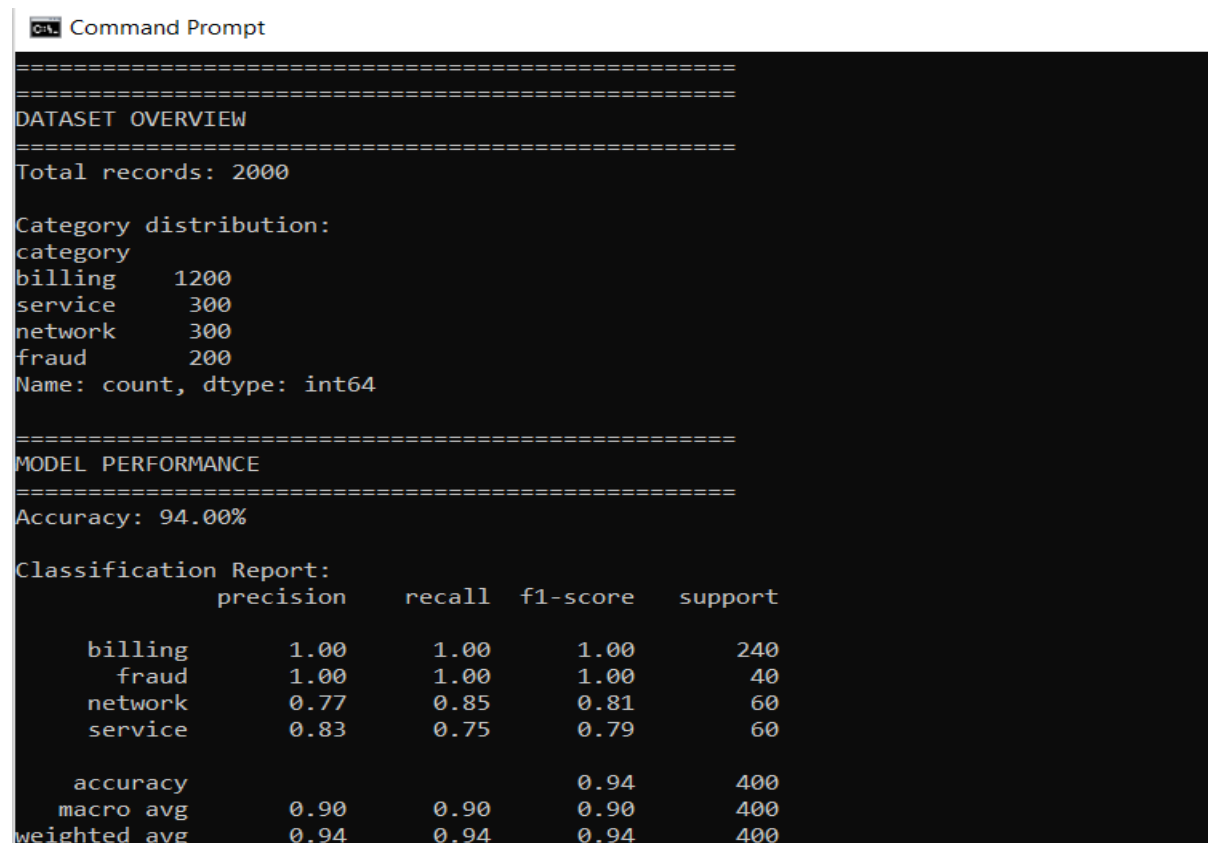
The dataset generator swapped some network and service labels. This introduced noise and created overlap between classes. After fixing leakage, both categories still show confusion because of label errors in the data.

9. Results

Model Accuracy

Accuracy: 94%

Classification Report



```
Command Prompt

=====
DATASET OVERVIEW
=====
Total records: 2000

Category distribution:
category
billing      1200
service      300
network      300
fraud        200
Name: count, dtype: int64

=====
MODEL PERFORMANCE
=====
Accuracy: 94.00%

Classification Report:
      precision    recall  f1-score   support

   billing       1.00      1.00      1.00        240
    fraud       1.00      1.00      1.00         40
  network       0.77      0.85      0.81         60
   service       0.83      0.75      0.79         60

   accuracy          0.94          400
  macro avg       0.90      0.90      0.90          400
weighted avg       0.94      0.94      0.94          400
```

```

Confusion Matrix:
[[240  0  0  0]
 [  0 40  0  0]
 [  0  0 51  9]
 [  0  0 15 45]]

Top Important Features:
-----
days                0.1060
account              0.0561
number              0.0329
acc                 0.0240
minutes             0.0210
network             0.0179
received            0.0175
charged             0.0162
getting             0.0160
speak               0.0158
consent             0.0157
using               0.0151
month               0.0130
coverage            0.0125
id                  0.0120

=====
Pipeline completed successfully
=====

```

Observation

The corrected model performed strongly because complaint categories contained clear keywords in the text. Remaining confusion between network and service was expected due to intentional label noise in the dataset.

10. Feature Importance Analysis

Top features included:

- days
- account
- network
- charged
- coverage
- consent

This confirmed the model was learning from real complaint language rather than leaked identifiers.

11. Key Lessons Learned

This challenge showed data leakage risks, correct preprocessing order, limits of accuracy under class imbalance, need for data understanding before training, and need for reliable evaluation methods.

12. Conclusion

The original classifier produced misleading performance due to data leakage and weak evaluation design. After fixing the pipeline, the model became more trustworthy, evaluation became realistic, and preprocessing followed correct machine learning practice. The final system generalizes better to unseen customer complaints and suits real-world use.